

Competitive Programming

Section 60: Final Capstone

Competitive Programming Course

2026-07-06

Table of contents

1	Final capstone	3
1.1	What “red-coder training” means	3
1.2	Capstone A: meet-in-the-middle	3
1.3	Capstone B: CRT	3
1.4	Capstone C: fast recurrence	4
1.5	Final self-review rubric	4
1.6	Capstone integration drill	4
1.7	Final implementation rule	4
1.8	Final practice protocol	5
1.9	What to maintain after the course	5
1.10	Final capstone self-test	5
2	Exercises	6

1 Final capstone

The end of the course is a launch point, not a finish line. Red-coder training means repeated cycles of contest, upsolve, proof cleanup, and template review. The capstone mixes subset search, congruence modeling, and recurrence speed.

1.1 What “red-coder training” means

At this level, improvement is less about learning isolated algorithms and more about reducing recognition time, proof time, and implementation risk.

A weekly cycle:

1. One serious contest or virtual contest.
2. Full upsolve of every missed problem near your range.
3. Rewrite one proof cleanly.
4. Stress-test one implementation.
5. Add one pattern to your notebook.

1.2 Capstone A: meet-in-the-middle

Maximum subset sum not exceeding X should trigger:

```
n around 40 -> split into two halves
enumerate sums -> sort one side -> binary search complement
```

The proof is that every subset is uniquely the union of a left-half subset and a right-half subset.

1.3 Capstone B: CRT

Merging congruences should trigger:

```
x = a mod m
x = b mod n
check gcd compatibility
merge into x = c mod lcm(m,n)
```

The compatibility check is not optional. If residues disagree modulo the gcd, there is no integer satisfying both.

1.4 Capstone C: fast recurrence

Many recurrence problems are solved by matrix exponentiation. Fibonacci has a special fast-doubling form:

```
(F(n), F(n+1)) -> (F(2n), F(2n+1))
```

The habit to keep is not memorizing Fibonacci only; it is recognizing when a linear recurrence needs logarithmic exponentiation.

1.5 Final self-review rubric

For each problem you solve after this course, ask:

- Did I identify the model before coding?
- Did I state the invariant or recurrence?
- Did I prove the transition?
- Did I test the smallest and largest meaningful cases?
- Did I preserve a reusable lesson for the next contest?

That loop is how this course continues after the final PDF.

1.6 Capstone integration drill

When opening a hard problem, write a three-column scratch table:

Observed constraint	Possible model	Why it might fit
$n \leq 40$	MITM	split exponential search
mod equations	CRT	merge congruences
linear recurrence	matrix/doubling	logarithmic query
tree all roots	rerooting	answer for every root
range updates	lazy segtree	update/query intervals

Then eliminate models whose assumptions fail. This habit is what turns a large course into contest speed.

1.7 Final implementation rule

For red-level problems, never start from the longest template. Start from the smallest correct skeleton that proves the model:

```
parse
build the compressed state
answer one query
test tiny cases
only then optimize or generalize
```

1.8 Final practice protocol

For the next 20 virtual contests:

```
Before contest: choose one focus skill.  
During contest: write model/invariant before each medium+ problem.  
After contest: upsolve until every official solution is understandable.  
After upsolve: rewrite one solution without looking.  
Weekly: review failed patterns and update the notebook.
```

The course gives you the map. This protocol turns the map into reflexes.

1.9 What to maintain after the course

Keep three artifacts:

```
template notebook: trusted implementation snippets  
pattern notebook: observations that unlock problems  
mistake notebook: bugs you have personally made
```

The mistake notebook matters most. Red-coder progress often comes from not making the same wrong assumption twice.

1.10 Final capstone self-test

Before considering the course complete, take one problem from each category and solve it without looking at the lesson:

```
Fenwick/segment tree query  
shortest path or flow model  
tree DP or rerooting  
string matching/hash  
number theory/combinatorics  
game or invariant problem
```

The target is not memorization; it is recognizing which model the statement is trying to hide.

2 Exercises

- A. Capstone Subset: Meet-in-the-middle: maximum subset sum not exceeding X .
- B. Capstone CRT: Merge pairs of congruences or report impossibility.
- C. Capstone Recurrence: Fast Fibonacci queries modulo $1e9+7$. # Contest checklist

State the invariant, choose the model, prove the operation, then code the smallest reliable version.